

Software Engineering in Industrial Practice (SEIP)

Dr. Ralf S. Engelschall

ARCHITECTURE FUNDAMENTALS

Communication Architectures

(Patterns)

TECHNISCHE UNIVERSITÄT MÜNCHEN

AF 08.2

UCT Unicast (one-to-one)

Communicate messages from one source to exactly one destination node. The destination node is explicitly and individually addressed.

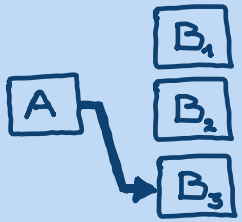
Rationale: private communication between exactly two nodes which both know each other beforehand.



ACT Anycast (one-to-any)

Communicate messages from one source to one of many destination nodes. The picked destination node usually is the network-topology-wise "nearest" or least utilized node in a group of nodes.

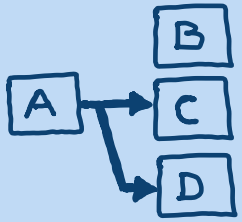
Rationale: Unicast, optimized for network failover scenarios, load balancing and CDNs.



MCT Multicast (one-to-many)

Communicate messages from one source to many destination nodes. The destination nodes usually form a group and are usually not individually addressed.

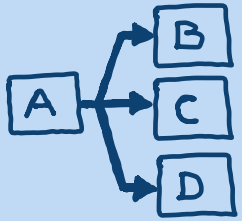
Rationale: node communication where destination nodes dynamically change or where total traffic should be reduced.



BCT Broadcast (one-to-all)

Communicate messages from one source to all available destination nodes. The destination nodes usually are implicitly defined by the extend of the local communication network segment.

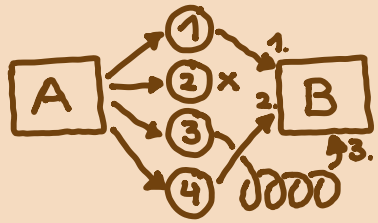
Rationale: spreading out messages to all available nodes for potential responses.



DGR Datagram (Single Packet)

Communicate messages as an unordered set of single packets, usually without any network congestion control, retries or other delivery guarantees.

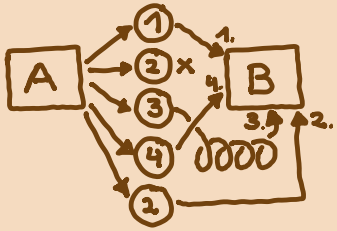
Rationale: simple low-overhead communication without prior communication establishment (handshake).



STR Stream (Sequence of Packets)

Communicate messages as an ordered sequence (stream) of packets, usually with network congestion control, retries and delivery guarantees (at-most-once, exactly-once, at-least-once).

Rationale: reliable communication between nodes.



PLL Pull (Request/Response, RPC)

Communicate by performing a request (from the client node) and pulling a corresponding response (from the server node).

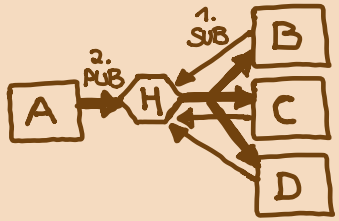
Rationale: Remote Procedure Call (RPC) like Unicast or Anycast communication.



PSH Push (Publish/Subscribe, Events)

Communicate by "subscribing" to "channels" of messages (on one or more receiver nodes or on an intermediate hub) once and then publishing events to those "channels" (on the sender node) multiple times.

Rationale: event-based Multicast or Broadcast communication.



Intellectual Content: Version 1.03 (2020-09-26) Authored 2018-2020 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.1 (2018-11-24) Copyright © 2018 Dr. Ralf S. Engelschall <http://engelschall.com> All Rights Reserved
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

ARCHITECTURE FUNDAMENTALS

Data Structure Architectures

(Patterns)

TECHNISCHE UNIVERSITÄT MÜNCHEN

AF 09.1

Data Structure Types

Scalar, Atom, Primitive Type

Plain integer or real number, single character or character string, not indexed and (for string only) accessed in O(1) by character position.

Tuple, Object, Structural Type, Record

Ordered, fixed-size sequence of scalar elements, each of individual type, indexed by name and accessed in O(1) by element name.

Sequence, Array, List

Ordered sequence of elements, each of same type, indexed by position and accessed in O(1) or O(n) by element position.

Set, Bag, Bucket

Unordered set of elements, each of same type, not indexed and accessed in O(1) or O(n) by element reference.

Map, Hash, Associative Array

Unordered sequence of elements, each of same type, indexed by (scalar) key and accessed in O(1) by key.

Graph, Nodes & Edges

Unordered set of linked elements (nodes), each of individual type, indexed by (scalar) key and accessed in O(1) by key or by following a directed link (edge).

Data Evolution Approaches

In-Place Editing

Modify data through direct in-place editing, overwriting the previous revision.

Stacking Revisions

Modify data through stacking revisions, preserving all previous revisions. Latest revision is always on top of stack.

Structural Difference

Modify data through merging, journaled domain-unspecific structural differences.

Operational Transformation (OT)

Modify data through applying journaled, domain-specific operational transformations.

Data Sharing Approaches

Event Sourcing & CRDT

Share data as a chronological sequence of data change events from which the data states can be (re)constructed. Optionally, use a Conflict-Free Replicated Data-Type (CRDT) protocol for the change events.

Ref.-Counting & Copy-on-Write

Share data between resources by using reference-counted data chunks, duplicating a chunk (and resetting its reference count to one) on write operations only and destroying a chunk once the reference count drops to zero.

Data Store Types

Key-Value Store

Storage of values in an unordered manner, indexed and queried by key.

Redis, Riak, MemcacheD, RocksDB, LevelDB

Large-Object Store

Storage of unstructured binary-large object (BLOB) data and its associated meta-data, indexed and queried by unique id.

Minio, SeaweedFS, AWS S3

Triple Store

Storage of subject-predicate-object triples, indexed and queried by subject/predicate/object values and example triples.

Redstore, Virtuoso

File-Tree Store

Storage of unstructured data as named files in a directory tree, indexed and queried by name path from root directory to leave file.

ZFS, XFS, UFS2, APFS

Graph Store

Storage of values as vertices and edges in a graph, both optionally referencing associated key/value pairs. Indexed and queried by key/value pairs and traversed by following edges.

Neo4J, OrientDB, ArangoDB

Document Store

Storage of structured "documents", indexed by id and key/value fields and queried by id and example documents.

MongoDB, CouchDB, RethinkDB

Relational/Table Store

Storage rows of fixed-size, typed value columns, indexed and queried by column values.

PostgreSQL, MariaDB, SQLite, H2, ORACLE DB, IBM DB2

Full-Text Store

Storage of unstructured text, indexed and queried by content words.

ElasticSearch, Solr, Groonga

Wide-Column Store

Distributed storage of rows of sparse (often untyped) value columns, indexed and queried by column values.

Cassandra, MonetDB, HBase, ScyllaDB

Time-Series Store

InfluxDB, MetricTank, RRDTool

Storage of integer or real values (y-axis) of a time-series (x-axis) into a fixed-size storage format in a round-robin manner where older values are increasingly aggregated (leading to lower resolutions at older times) and finally overwritten.

DataVault Store

Long-term historical storage of foreign, arbitrary relational data in a fixed schema of hubs, links and satellites, indexed and queried for analysis and reporting purposes.

DataVault 2.0

BlockChain Store

Storage of values in an unordered manner within information blocks which are cryptographically chained through their hash values and distributed in a peer-to-peer way.

Ethereum, Quorum, Tendermint, Hyperledger

Intellectual Content: Version 1.1.3 (2020-09-26) Authored 2011-2020 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.1.2 (2025-07-28) Copyright © 2012-2025 Dr. Ralf S. Engelschall <http://engelschall.com> All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

ARCHITECTURE FUNDAMENTALS

Data Persistence Architectures

(Patterns)

TECHNISCHE UNIVERSITÄT MÜNCHEN

AF 09.2

Data Guarantees

CAP (Trade-In)

A distributed data store cannot provide more than two out of three guarantees: Consistency (C), Availability (A), Partition-Tolerance (P). So, it has to choose between Consistency (CP) and Availability (AP) when a network partition or failure happens.



BASE (NoSQL)

The semantics (usually of NoSQL systems) of (B)asically (A)vailable, (S)oft state, and (E)ventual consistency. BASE systems favor Availability over Consistency in the CAP-context.



ACID (RDBMS, NewSQL)

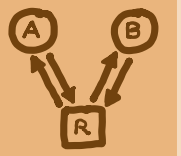
The four guarantees provided in parallel (usually by RDBMS and NewSQL systems): Atomicity, Consistency, Isolation and Durability. ACID systems usually favor Consistency over Availability in the CAP-context.



Data Access

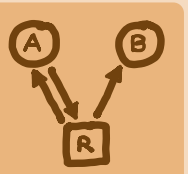
Shared Read/Write

Shared access to data for both read and write operations. Example: Multiple threads on heap or Master-Master database setup.



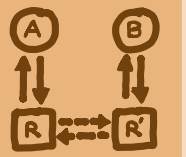
Shared Read / Exclusive Write

Shared access to data for read operations and exclusive access (via a single "owning" component) to data for write operations. Example: RDBMS Master-Slave cluster with shared storage.



Shared Nothing

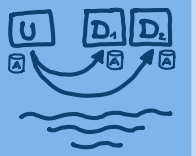
No shared access to data at all for both read and write operations. Example: Leader-Follower setup with RAFT consensus where Leader writes data only.



Data Spreading & Aggregation

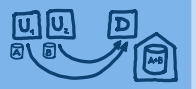
Data River (1-to-N)

A real-time fan-out replication of data from a single upstream/source data repository to multiple downstream/target data repositories.



Data Mart (N-to-1), ODS

A massive sized, easily accessible data repository for storing "big data" from many upstream sources in a (real-time and) structured way and with knowing the actual subsequent analysis usage.



Data Lake (N-to-1), Cache

A massive sized, easily accessible data repository for storing "big data" from many upstream sources in a (real-time) semi-structured way and without knowing the actual subsequent usage.



Data Access Grouping

Transaction

Protect a sequence of operations from interim exceptions by bracketing the operations in a technical transaction (ensuring that either all or none of the operations succeed).



Compensation

Protect a sequence of operations from interim exceptions by undoing the already succeeded operations through domain-specific compensating (reverse) operations.



Data Consistency

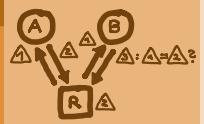
Exclusive Locking (Mutex)

Protect data from concurrent access and resulting inconsistencies with a mutual exclusion lock (mutex) which allows just a single peer to access the data at a time.



Optimistic Locking

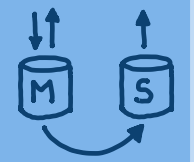
Protect data from concurrent access and resulting inconsistencies by taking note of a revision number or content hash during read operations and checking that this information has not changed before writing the data.



Data Transfer

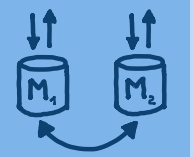
Replication

Continuously stream or regularly copy data from a master system to one or more slave systems in order to either read the data from slave systems faster or have slave systems available as a fallback/ backup in case of a failure of the master system.



Synchronization

Continuously stream or regularly copy data between multiple master systems and resolve potential concurrent data modification conflicts. This way allow distributed and even disconnected computing.



Intellectual Content: Version 1.02 (2019-04-12), Authored 2018-2019 by Dr. Ralf S. Engelschall <http://engelschall.com>. All Rights Reserved.
Graphical Illustration: Version 1.1.0 (2022-11-06). Copyright © 2018-2022 Dr. Ralf S. Engelschall. Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

ARCHITECTURE FUNDAMENTALS

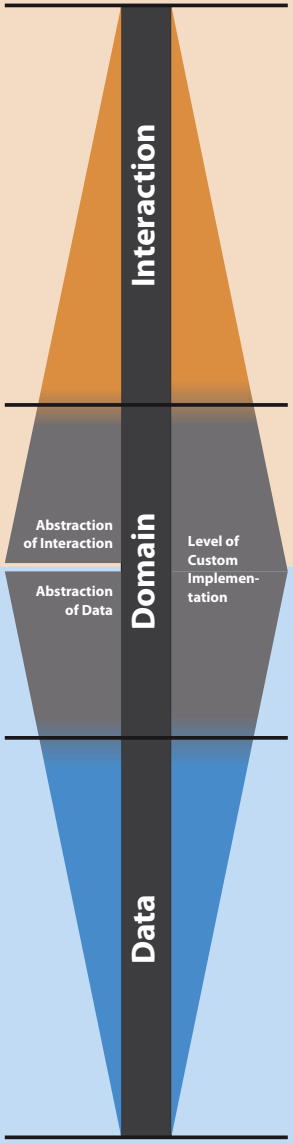
Application Reference Architecture

TECHNISCHE UNIVERSITÄT MÜNCHEN

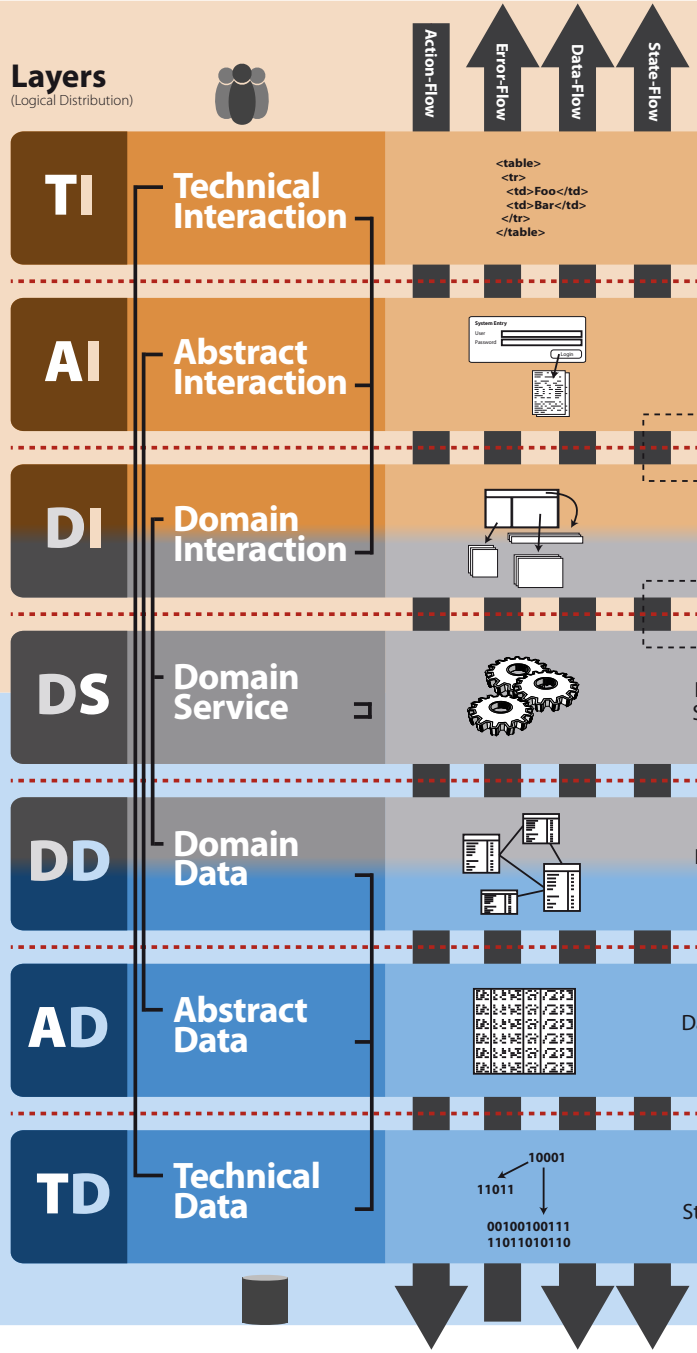
AF 10.1

Intellectual Content: Version 1.0.9 (2022-10-09), Authored 2009-2022 by Matthias Bruckey/ins, Andreas Martens and Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.9 (2022-10-09), Copyright © 2012-2022 Dr. Ralf S. Engelschall <http://engelschall.com> All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

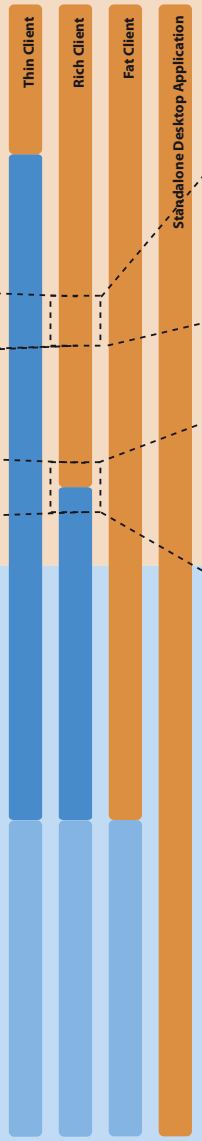
Focus
(Abstraction & Implementation)

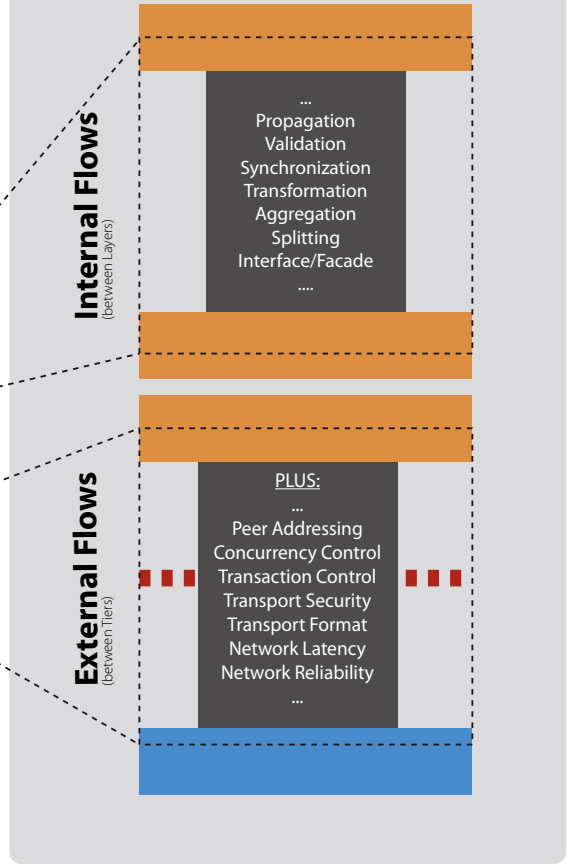


Layers
(Logical Distribution)



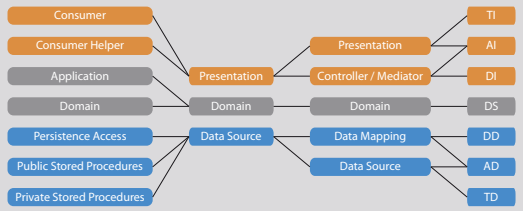
Tiers
(Physical Distribution)





Layer Classification Comparison:

[1] Andreas Martens: Architekturbasiertes Vorgehensmodell zur Identifizierung und Lokalisierung von Architektur-Kriterien in Enterprise-Anwendungen, Universität Paderborn, 2010.
[2] Brown et al.: Enterprise Java Programming with IBM Websphere, Addison-Wesley, 2001.
[3] Martin Fowler: Patterns of Enterprise Application Architecture, Addison-Wesley, 2003.
[4] Nilsson: .NET Enterprise Design with Visual Basic .NET and SQL Server 2000, Sams, 2002.



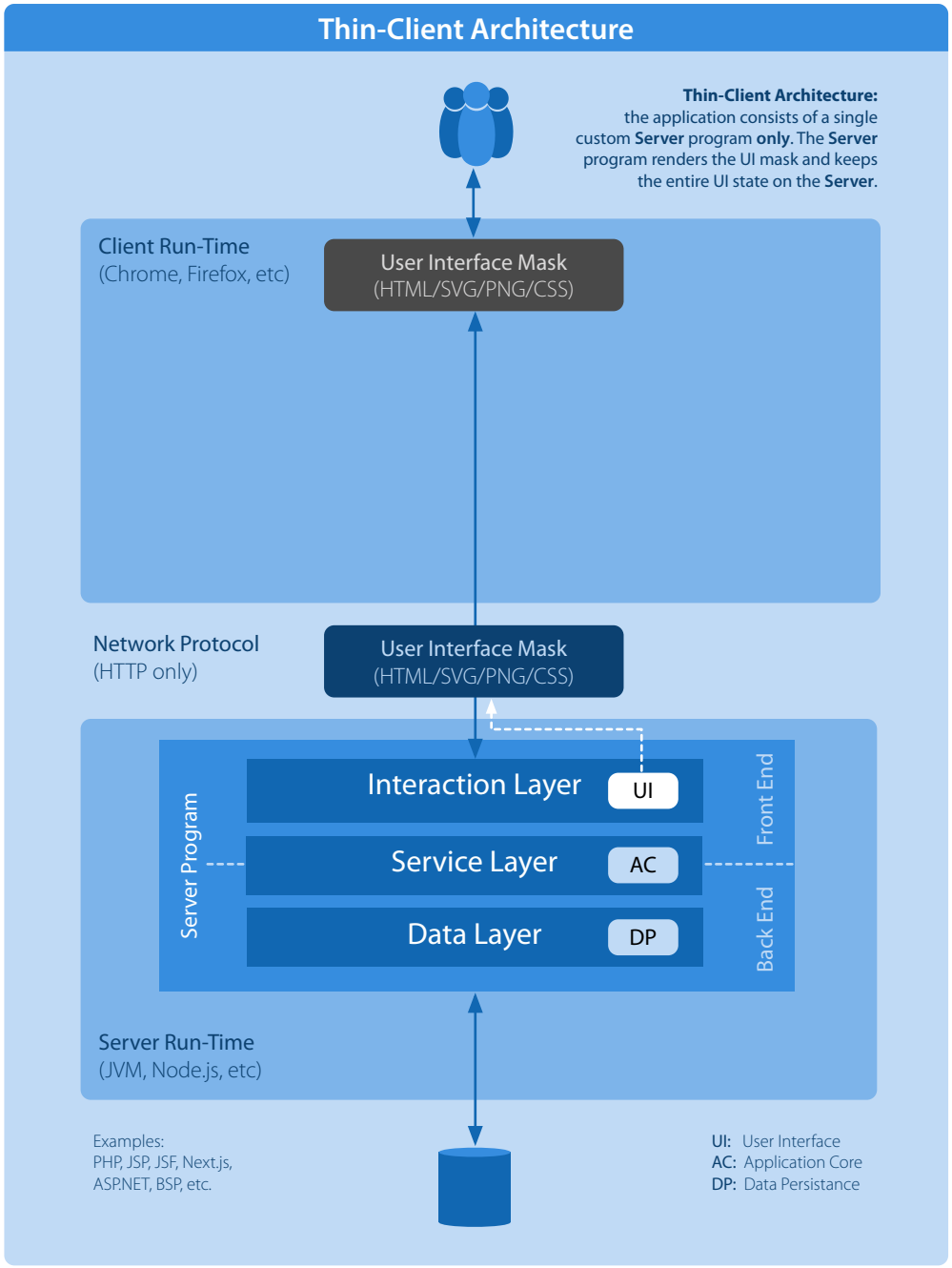
ARCHITECTURE FUNDAMENTALS

Client-Server Architecture

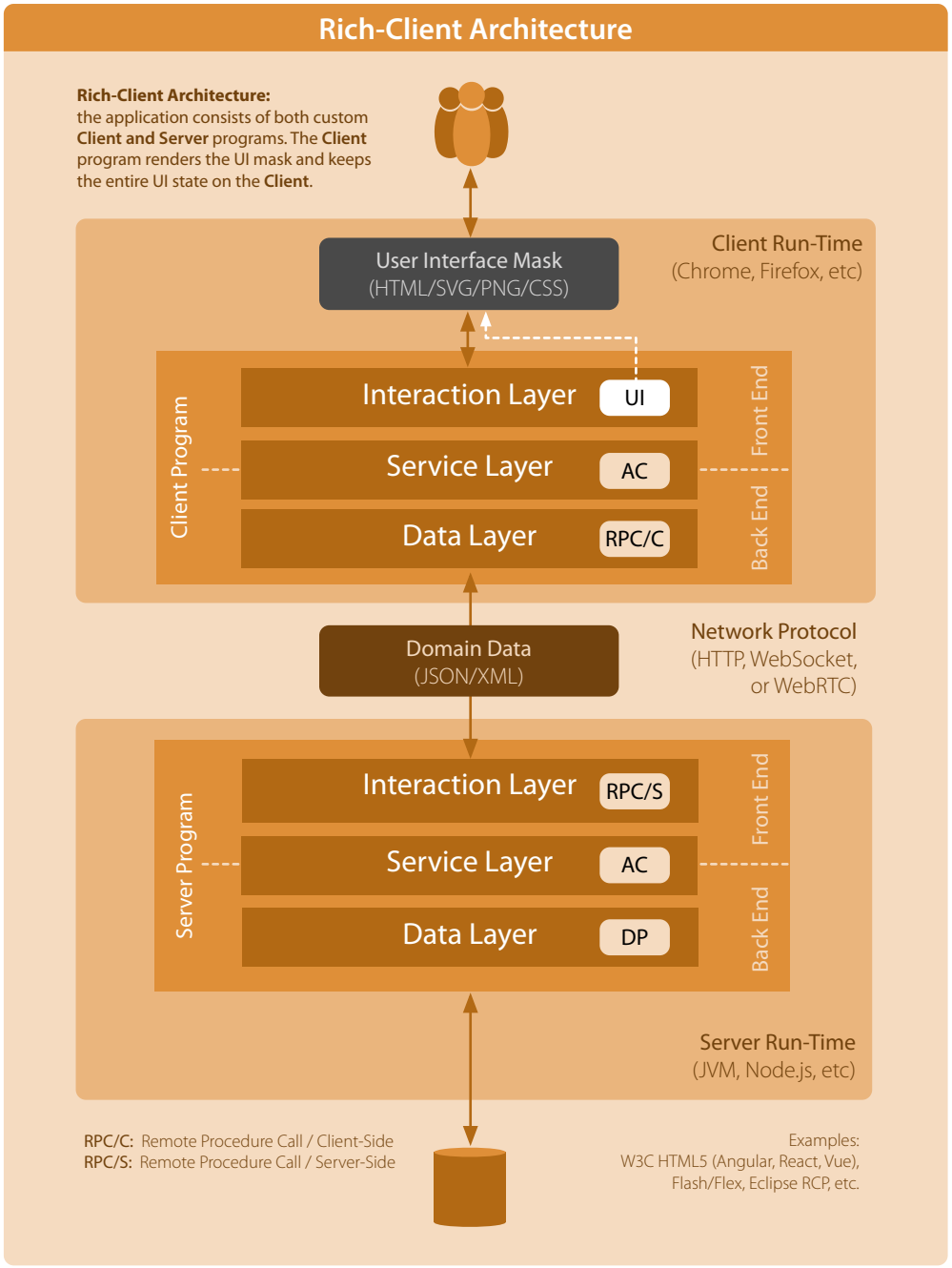
TECHNISCHE UNIVERSITÄT MÜNCHEN

AF 10.2

Thin-Client Architecture



Rich-Client Architecture



Intellectual Content: Version 1.0.0 (2021-09-14) Authored 2021 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.0 (2021-09-14) Copyright © 2021 Dr. Ralf S. Engelschall <http://engelschall.com> All Rights Reserved
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

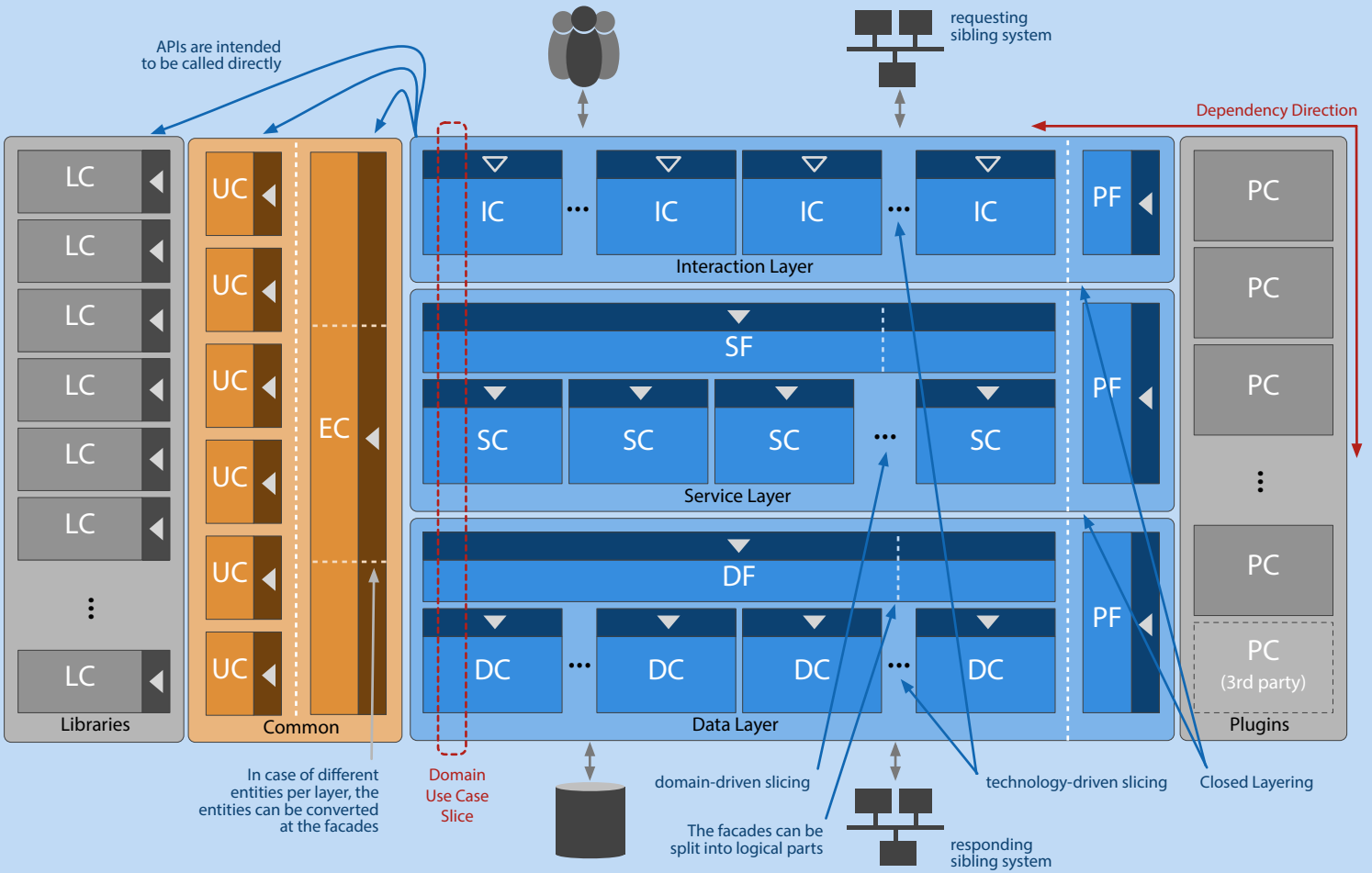
ARCHITECTURE FUNDAMENTALS

Information System Architecture

TECHNISCHE UNIVERSITÄT MÜNCHEN

AF 10.3

Reference Architecture Blueprint



Component Types:

IC: Interaction Component
SF: Service Facade
SC: Service Component
DF: Data Facade
DC: Data Component
PF: Plugin Facade
PC: Plugin Component
UC: Utility Component
EC: Entity Component
LC: Library Component

Service Facade (SF) Potential Functionality:
Request Validation, Use-Case Authorization, Process Orchestration (Dispatching), Data Splitting/Aggregation/Conversion, Transaction Handling, Run-Time Use-Case Tracing, Result Caching, etc.

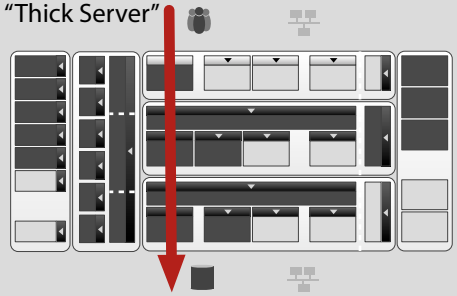
Data Facade (DF) Potential Functionality:
Data Access Authorization, Data Orchestration (Dispatching), Data Splitting/Aggregation/Conversion, Transaction Handling, Run-Time Data Access Tracing, Data Caching, etc.

Input/Output Interface
Application Programming Interface (API)

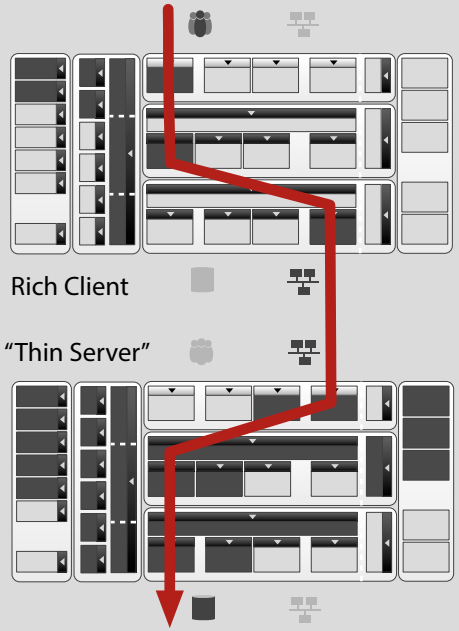
Blueprint Usage

Example Scenario 1

Thin Client / "Thick Server"



Example Scenario 2



Intellectual Content: Version 1.3.5 (2023-07-15), Authored 2007-2023 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.2.3 (2023-10-09), Copyright © 2011-2023 Dr. Ralf S. Engelschall <http://engelschall.com>. All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.